

```
#include <stdio.h>
#include <stdlib.h>

int global = 100;          // Global segment (.data)
static int s = 50;         // Static segment (.data)

int main()
{
    int stack = 10;        // Stack segment

    int *heap = (int *)malloc(sizeof(int)); // Heap segment
    *heap = 20;

    printf("Memory Address Layout of Process\n\n");

    printf("Code Segment Address   : %p\n", (void *)main);
    printf("Global Variable Addr   : %p\n", (void *)&global);
    printf("Static Variable Addr    : %p\n", (void *)&s);
    printf("Heap Variable Address   : %p\n", (void *)heap);
    printf("Stack Variable Addr     : %p\n", (void *)&stack);

    free(heap);

    return 0;
}
```

```
nlr2007@NLR:~$ nano memoryseg.c
nlr2007@NLR:~$ gcc memoryseg.c -o ms
nlr2007@NLR:~$ ./ms
Memory Address Layout of Process

Code Segment Address   : 0x605aa280f1c9
Global Variable Addr   : 0x605aa2812010
Static Variable Addr   : 0x605aa2812014
Heap Variable Address   : 0x605aa90de2a0
Stack Variable Addr     : 0x7fff3a30f9bc
```

2.

```
nlr2007@NLR:~$ cat valgrind.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    // Memory allocation using malloc()
    a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("malloc() allocated memory for 5 integers\n");

    // Assign values
    for(int i = 0; i < 5; i++)
    {
        a[i] = i + 1;
    }

    // Memory allocation using calloc()
    b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("Memory allocation failed\n");
        free(a);
        return 1;
    }

    printf("calloc() allocated memory for 5 integers\n");

    // Memory reallocation using realloc()
    a = (int *)realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
        printf("Reallocation failed\n");
        free(b);
        return 1;
    }

    printf("realloc() increased memory size to 10 integers\n");

    // Display allocated memory
    printf("\nValues in malloc/realloc memory:\n");

    for(int i = 0; i < 10; i++)
    {
        a[i] = i + 10;
        printf("%d ", a[i]);
    }

    printf("\n\nValues in calloc memory:\n");

    for(int i = 0; i < 5; i++)
    {
        printf("%d ", b[i]);
    }

    // Free allocated memory
    free(a);
    free(b);

    printf("\n\nMemory released successfully using free()\n");

    return 0;
}
```

```

nrlr2007@NLR:~$ nano valgrind.c
nrlr2007@NLR:~$ gcc valgrind.c -o vd
nrlr2007@NLR:~$ ./vd
malloc() allocated memory for 5 integers
calloc() allocated memory for 5 integers
realloc() increased memory size to 10 integers

Values in malloc/realloc memory:
10 11 12 13 14 15 16 17 18 19

Values in calloc memory:
0 0 0 0 0

Memory released successfully using free()

```

3.

```

GNU nano 7.2 COW.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int x = 100;

    printf("Before fork(), x = %d\n", x);

    pid_t pid = fork();

    if(pid == 0)
    {
        // Child process
        printf("Child Before Modification: x = %d\n", x);

        // Modifying data causes Copy-on-Write
        x = 500;

        printf("Child After Modification: x = %d\n", x);
    }
    else if(pid > 0)
    {
        // Parent process
        sleep(2);

        printf("Parent Process: x = %d\n", x);

        wait(NULL);
    }
    else
    {
        printf("Fork failed\n");
    }

    return 0;
}

```

```

1 nrlr2007@NLR:~$ gcc COW.c -o c
nrlr2007@NLR:~$ ./c
Before fork(), x = 100
Child Before Modification: x = 100
Child After Modification: x = 500
Parent Process: x = 100
nrlr2007@NLR:~$ |

```